GymFlow

Gym management and class booking system

Requirements, use cases and class diagrams

Óscar Muñoz, Víctor Lorenzo, Pablo Alpuente, Diego Fuentes, Jorge Terres, Alejandra Zurdo

Software Engineering 1 – Warsaw University of Technology, 2026/2027

1 System overview

GymFlow manages the operation of a fitness club: class timetable, bookings with capacities and waiting lists, room allocation, and coordination between members, trainers and administrative staff. It consists of three autonomous agents that communicate only through structured network requests (REST/JSON). The **Admin/Server Agent** is the central coordinator: it validates and routes every request, so members and trainers never talk directly to each other (Figure 1). A Scheduler triggers periodic jobs (membership expiry, reminders).

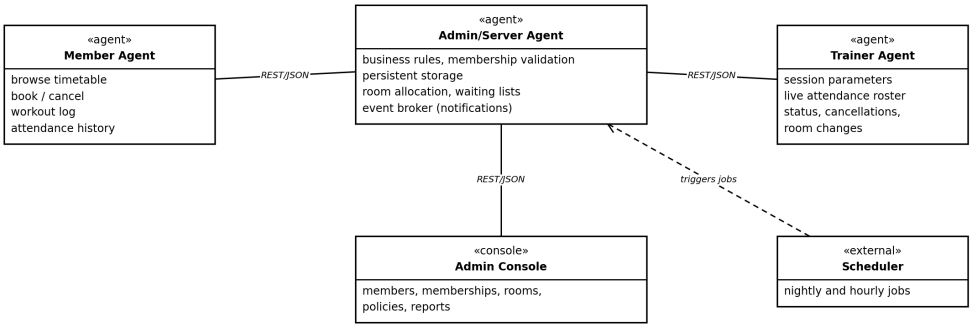


Figure 1: Agents and communication flow: Member Agent ↔ Admin/Server Agent ↔ Trainer Agent.

Services of the Admin/Server Agent (communication between the modules goes through them):

Service	Main operations	Called by	Use cases
AuthService	login, authorize	all agents	S01
MembershipService	validate, subscribe, pause, expireDue	Booking, Admin Console, Scheduler	S02, S10
BookingService	book, cancel, joinWaitlist, promoteWaitlist	Member Agent, Admin Console	S03, S04
ScheduleService	createSession, reschedule, cancelSession, hasConflict	Trainer Agent, Admin Console	S06, S07
AttendanceService	roster, mark	Trainer Agent	S08
WorkoutLogService	add, list	Member Agent	S09
NotificationService	send, retryFailed	Booking, Schedule, Scheduler	S05, S11

2 User stories

Format: *As a [role], I want [goal], so that [benefit].* Priorities follow MoSCoW (M = Must, S = Should, C = Could); 19 stories are Must, 6 Should and 1 Could. The last column links each story to the use cases (UC-) of Section 3. Out of scope for this version (Won't have): online payments (memberships are sold and paid at the gym), multi-branch gyms, wearable integration, door/turnstile hardware.

ID	As a	I want to	So that	Prio	UC
US-01	User	log in with my account	my data is private and my role is known	M	M01, T01, A01, S01
Member Agent					
US-02	Member	see my membership status and edit my profile	I know when to renew and the gym has correct data	M	M02, M11
US-03	Member	browse the timetable by discipline, trainer and day, with live free places	I can plan my week	M	M03
US-04	Member	book a place in a class	I am sure I have a spot	M	M04, M05, S02, S03
US-05	Member	cancel a booking	the place is free for someone else	M	M06, S03
US-06	Member	join the waiting list of a full class and be promoted automatically	I do not miss the class if somebody cancels	M	M07, S04
US-07	Member	be notified of cancellations, room changes, promotions and reminders	I do not travel for nothing	M	M10, S05
US-08	Member	see my attendance history and keep a workout log	I can follow my progress	S	M08, M09, S09
Trainer Agent					
US-09	Trainer	see my weekly schedule	I know when and where I teach	M	T02
US-10	Trainer	create a session with discipline, time and maximum participants	members can book it	M	T03, T04, S06, S07
US-11	Trainer	edit or reschedule my sessions	I can react to changes in my availability	S	T05, S06
US-12	Trainer	cancel a session and have all booked members notified	nobody comes to a class that does not take place	M	T06, T07, S05
US-13	Trainer	see the live roster and waiting list and send an announcement to participants	I know who to expect and can give instructions	M	T08, T11
US-14	Trainer	mark who attended and who did not	attendance is recorded correctly	M	T09, S08
US-15	Trainer	report session status (started, delayed, finished) or a room change	members and staff see the real situation	M	T10, S05
Administration console					
US-16	Administrator	manage member accounts and their memberships	only members with a valid membership can book	M	A02, A03
US-17	Administrator	manage trainer accounts	only authorised people teach and see rosters	M	A04
US-18	Administrator	manage rooms, equipment and disciplines	rooms are not double-booked and the offer is consistent	M	A05, A06
US-19	Administrator	move or cancel any session and notify those affected	I can solve emergencies and conflicts	S	A07, A12
US-20	Administrator	configure gym policies (cancellation window, waiting-list size, opening hours)	rules change without a new release	S	A08
US-21	Administrator	see, export and trace occupancy/attendance reports and the event log	I can plan timetable and staff	S	A09, A10, A11
Server Agent and Scheduler (non-human actors)					
US-22	Server Agent	validate the membership at every booking	only valid members take places	M	S02, S03
US-23	Server Agent	assign rooms and reject sessions overlapping in room or trainer	nothing is double-booked	M	S07
US-24	Server Agent	dispatch events between members and trainers and promote waitlisted members	every party sees the same state	M	S04, S05
US-25	Scheduler	expire memberships after their end date every night	expired members cannot book	S	S10
US-26	Scheduler	trigger class reminders every hour	members are reminded in time	C	S11

3 Use case diagrams

One diagram per module. Logging in is a precondition of every client use case. When one agent calls another, the call is a service use case of the Admin/Server Agent (UC-S..) and the calling agent appears as a non-human actor.

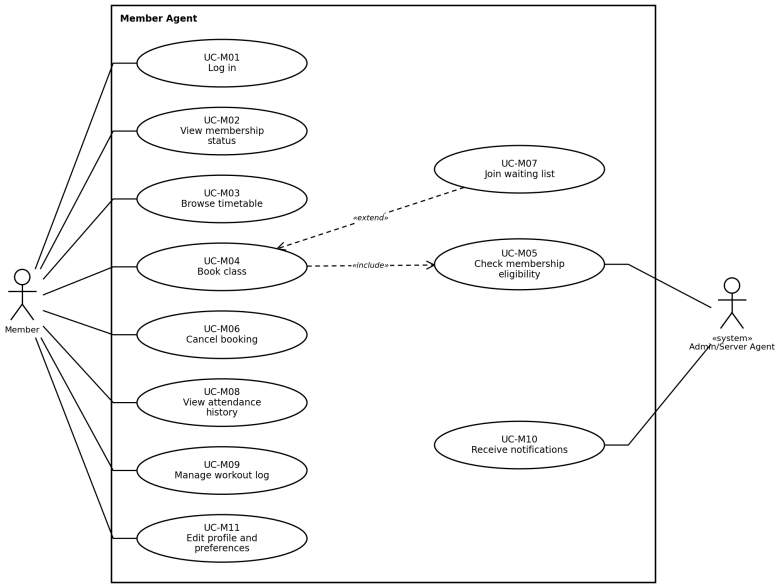


Figure 2: Use cases: Member Agent.

Member Agent. The Member is the only human actor. Booking (M04) always includes the eligibility check (M05), done by the Server; joining the waiting list (M07) only happens when the class is full, so it extends booking. The Admin/Server Agent appears as a non-human actor because it performs the check and pushes notifications (M10).

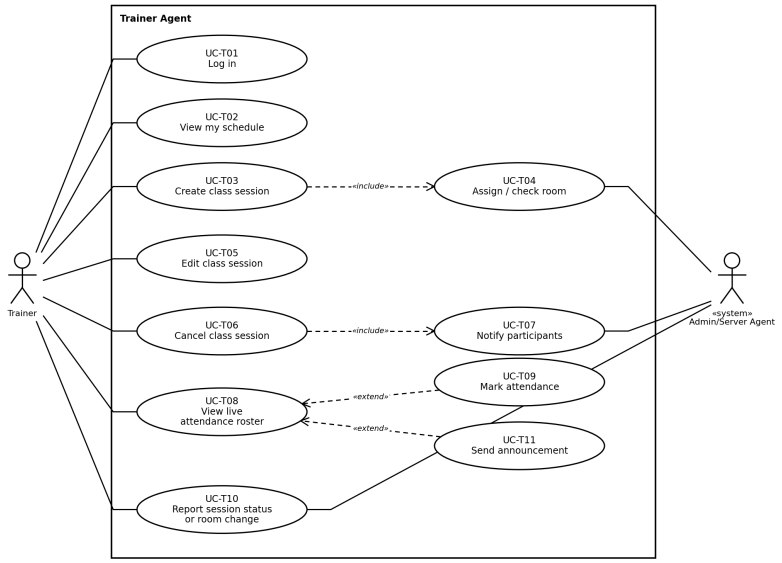


Figure 3: Use cases: Trainer Agent.

Trainer Agent. Creating a session includes the room check (T04); cancelling a session includes notifying participants (T07). Marking attendance (T09) and sending announcements (T11) both start from the live roster (T08), so they extend it.

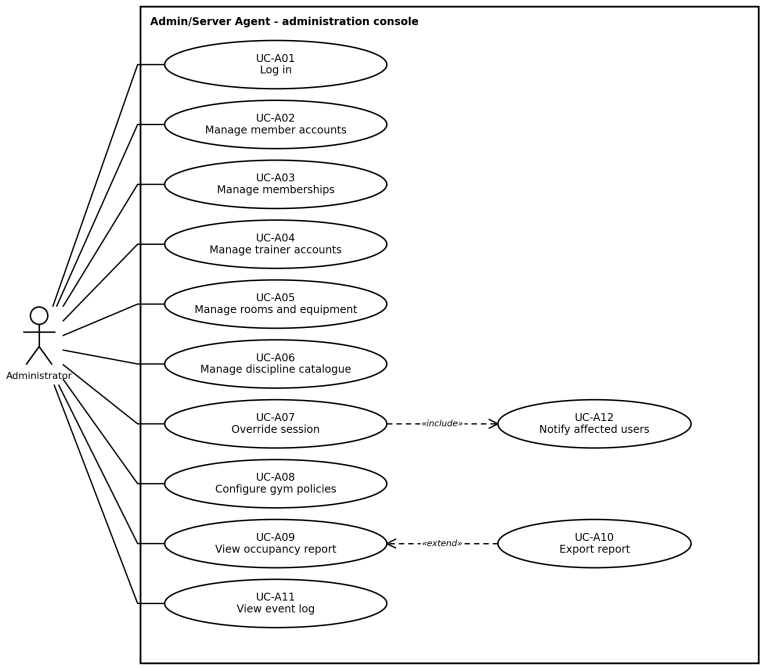


Figure 4: Use cases: administration console.

Administration console. The Administrator manages accounts, memberships, rooms, disciplines and policies. Overriding a session (A07) always includes notifying the affected users (A12); exporting a report (A10) extends the occupancy report (A09).

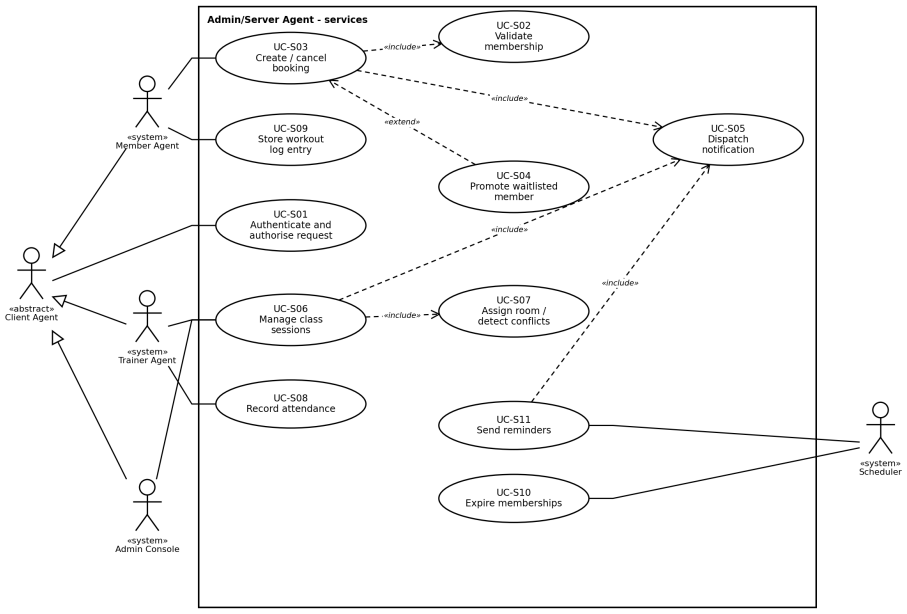


Figure 5: Use cases: services of the Admin/Server Agent and non-human actors.

Server services. This is what the other modules actually call. Member Agent, Trainer Agent and Admin Console are non-human actors that generalise to one abstract Client Agent, because all authenticate the same way (S01). The Scheduler is the non-human actor of the nightly expiry (S10) and hourly reminders (S11). Promoting a waitlisted member (S04) extends booking (S03).

4 Crucial requirements (Cockburn's schema)

Three use cases that involve the communication between the agents were chosen as crucial: booking (Member → Server), cancelling a session (Trainer → Server → Members) and promoting a waitlisted member (Server).

UC-M04 Book class

Goal level	User goal
Stakeholders and interests	Member wants a guaranteed place; Trainer wants a correct attendee list; Administrator wants no overbooking and plan limits respected.
Primary actor	Member (supporting: Admin/Server Agent)
Preconditions	Member is logged in; the session is SCHEDULED and has not started.
Trigger	Member taps "Book" on a session of the timetable.
Main scenario	1. Member Agent sends `bookClass(sessionId)` to the Server. 2. Server checks eligibility (UC-S02): active membership on the class date, plan allows the discipline, weekly limit, no overlapping booking. 3. Server checks that the session has a free place. 4. Server creates the Booking as CONFIRMED in one transaction. 5. Server dispatches a confirmation (UC-S05); Member Agent shows it in "My bookings".
Extensions	2a. Membership missing, expired or limit exceeded: request rejected with the reason. 3a. Session full: Server offers the waiting list (UC-M07); the booking is created as WAITLISTED, or rejected if the list is full. 3b. Two members ask for the last place at once: the transaction orders them; the second follows 3a. 5a. Notification fails: the booking stays valid and the message is retried.
Postcondition	A Booking exists; confirmed bookings never exceed the session capacity. If the request fails nothing is created.

UC-T06 Cancel class session

Goal level	User goal
Stakeholders and interests	Trainer wants to react quickly (e.g. when ill); Members do not travel for a cancelled class and keep their credits; Administrator is informed about late cancellations.
Primary actor	Trainer (supporting: Admin/Server Agent)
Preconditions	Trainer is logged in and leads the session; it is SCHEDULED and has not started.
Trigger	Trainer chooses "Cancel session" in the schedule.
Main scenario	1. Trainer enters a reason; Trainer Agent shows how many members are booked or waiting and asks for confirmation. 2. Trainer confirms; Trainer Agent sends `cancelSession(id, reason)`. 3. Server sets the session to CANCELLED, cancels all bookings and waiting-list entries and restores the session credits. 4. Server notifies all affected members (UC-T07 / S05). 5. Trainer Agent shows "N members notified".
Extensions	1a. Session starts in less than 2 hours: the Administrator is notified as well. 1b. Session already started or finished: cancellation not allowed. 3a. Cancelling the bookings fails: everything is rolled back and an error is shown. 4a. Some notifications fail: they are retried.
Postcondition	Session and all its bookings are CANCELLED, credits restored, members notified; either everything is cancelled or nothing.

UC-S04 Promote waitlisted member

Goal level	Sub-function (system level)
Stakeholders and interests	Member gets a place without checking the app; Trainer wants classes as full as possible; Administrator wants a fair first-in-first-out waiting list.
Primary actor	Server Agent
Preconditions	The session is SCHEDULED, has a non-empty waiting list and a place has become free.
Trigger	BookingService detects a free place in a session with waitlisted bookings.
Main scenario	1. Server takes the WAITLISTED booking with the lowest position. 2. Server validates the member's membership again for the class date (UC-S02). 3. Server sets the booking to CONFIRMED and renumbers the waiting list. 4. Server notifies the promoted member (UC-S05).
Extensions	1a. Waiting list empty: the place stays free. 2a. Membership no longer valid: the booking is CANCELLED and the Server continues with the next one. 4a. Notification fails: the promotion stays valid and the message is retried.
Postcondition	Confirmed bookings do not exceed the capacity; waiting-list positions are consecutive.

5 Class diagrams

Figure 6 is the domain model (data stored by the system); Figure 7 shows the three agents, the services of the Admin/Server Agent and the external Scheduler. Only the important attributes and operations are shown; simple types (Money, DateRange, DateTime, UUID, Schedule, enumerations) are used as field types without their own box.

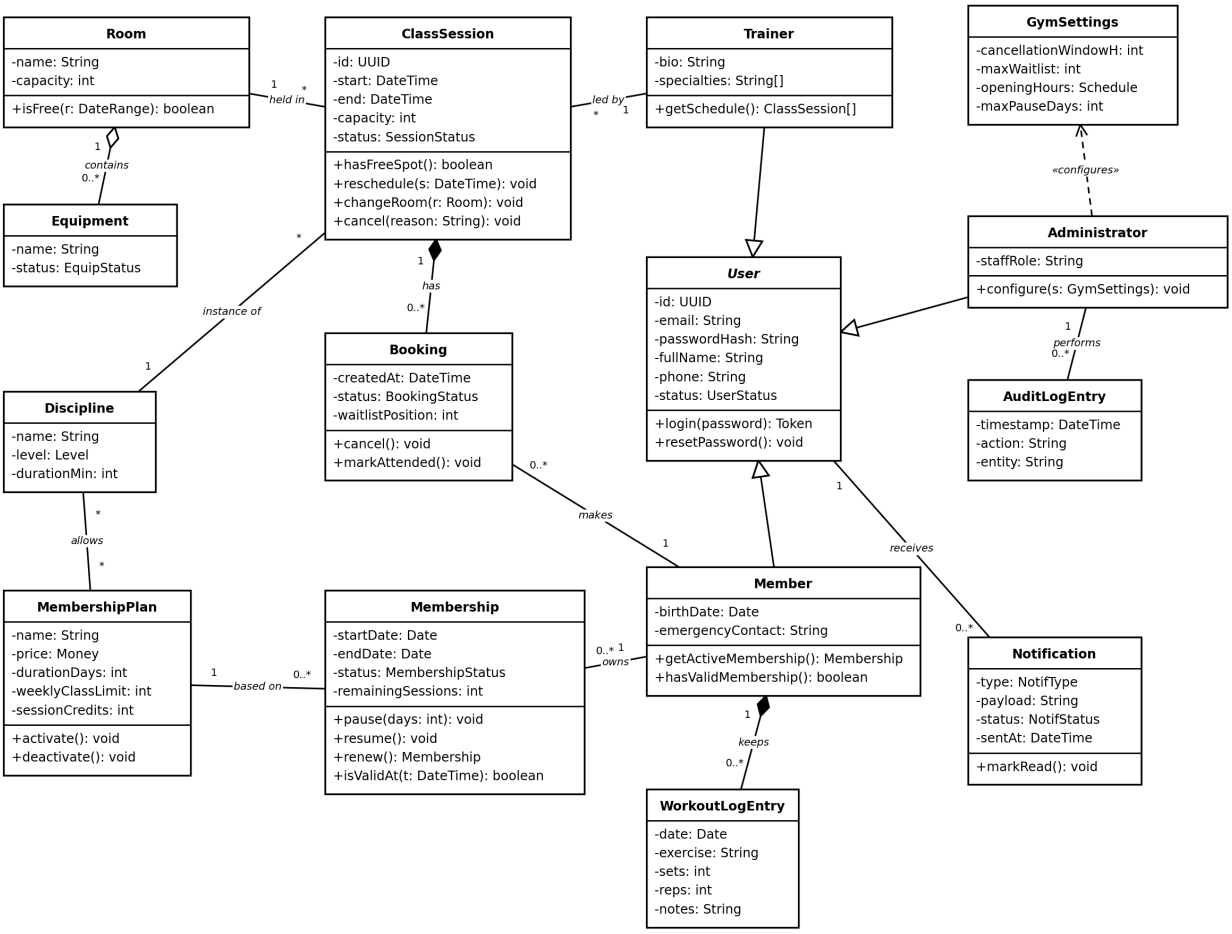


Figure 6: Class diagram: domain model.

Domain model. User is abstract and is extended by Member, Trainer and Administrator, matching the actors of the use cases. MembershipPlan (what the gym offers) is separate from Membership (what a member owns). A ClassSession has a Room and a Trainer, which is what the Server needs to detect conflicts (S07), and owns its Bookings (composition), which is where capacity and the waiting list are controlled (M04, S04). GymSettings holds the policies read by the services.

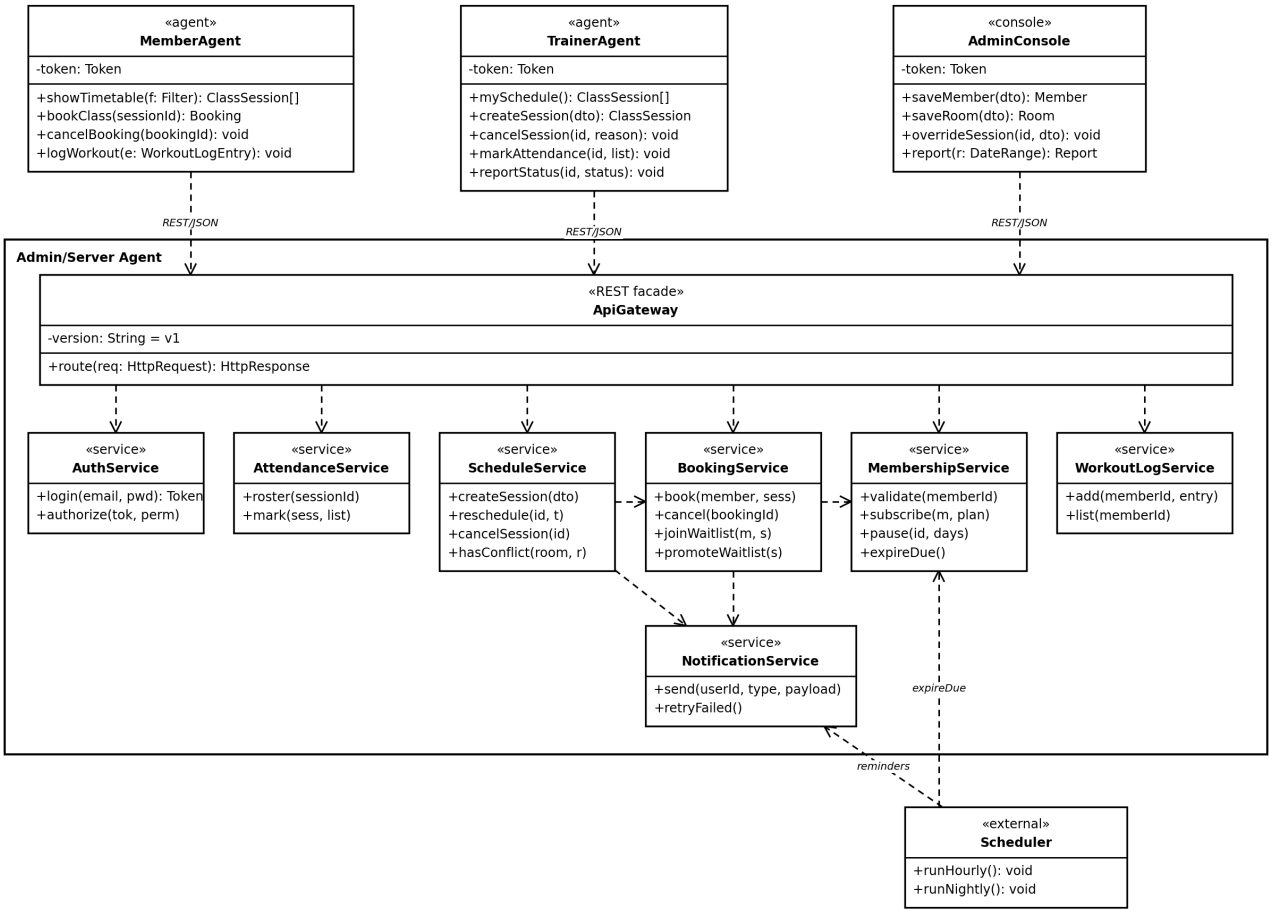


Figure 7: Class diagram: agents, services and external systems.

Agents and services. The three clients only know the ApiGateway (REST facade, API v1); each service can change internally without changing the agents. BookingService calls MembershipService to validate and NotificationService to inform; ScheduleService calls BookingService when a session is cancelled. Together, the two diagrams cover every module of the use case diagrams: Member Agent, Trainer Agent, Admin Console, the seven services and the Scheduler.